

A CUDA Silent-Fallback Bug in CTranslate2 + PyInstaller

Anchored in the BabelGG v2 codebase at `D:\AbeSoft\BabelGG_v2` .
Benchmarks collected 2026-08-05 on the hardware listed in §5.1.
All code references cite `file:line` locations verified against the working tree.

Abstract

We report two engineering findings from building BabelGG v2, a frozen-Windows-executable deployment of NLLB-200 distilled 600M via CTranslate2 with int8_float16 quantization on GPU. Both findings are previously undocumented and operationally significant.

Finding 1: Silent GPU-to-CPU fallback in PyInstaller

deployments. When CTranslate2 is initialized inside a frozen PyInstaller Windows executable, and the bundled NVIDIA runtime DLLs (`cublas64_12.dll` , `cublasLt64_12.dll` , `cudart64_12.dll`) are not discoverable through the Windows DLL search path at process start, the CTranslate2 translator falls back to CPU silently — no exception, no log entry on the user-facing side, no error surfaced through `ctranslate2.get_cuda_device_count()` . The fallback is invisible at the API level and produces correct translations at substantially degraded latency. We isolate the root cause to the PyInstaller frozen-executable DLL search path and present a two-component fix: a PyInstaller runtime hook (`rthook_dlls.py` , 11 lines) plus an explicit `os.add_dll_directory` enumeration (`core/hardware.py:17-68`) called before any CTranslate2 import. A two-stage verification system — preflight (`core/flash.py:511-526`) plus runtime CUDA-error recovery (`core/flash.py:617-644`) — catches the failure mode at both startup and runtime.

Finding 2: Source-language detection limits practical coverage.

The engine's source-language detector, based on Unicode charset analysis plus a small marker-word heuristic and the `langdetect` library, misidentifies a substantial fraction of European-language inputs as English. Spanish and German sentences that lack the heuristic marker words are flagged as English roughly 80% of the time; the engine then short-circuits with `src == tgt` and returns `None` without invoking the translation model. Direct calls to CTranslate2 on the same rejected inputs produce valid translations in every case we tested, so the model is capable and the detection pipeline is the bottleneck. The empirical failure rates we observed (50-85% on European → English) are dominated by detection failures, not model quality. We integrate a small fix —

`fasttext-langdetect` as a pre-detection step — that drops the German → English failure rate from 85% to 0%, Spanish → English from 75% to 10%, and overall detection accuracy from 51% to 86% on the benchmark corpus.

Measured characteristics on a workstation with an NVIDIA RTX 3080 (10 GB) and CTranslate2 4.7.1: the `int8_float16` NLLB-200 distilled 600M model loads in ~11 s cold and runs at a median latency of approximately 67 ms per translation across a 10-language-pair matrix when the call succeeds. The CPU fallback path (int8 quantization on the same model) achieves a median latency of approximately 250 ms — roughly 3.7× slower than GPU on the same task, far less than the 10× penalty commonly assumed.

Both findings matter for any developer shipping CTranslate2-based translation products through PyInstaller on Windows, and both provide small, well-localized fixes that do not require modifying CTranslate2 itself.

1. Introduction

CTranslate2 [OpenNMT, 2023] is the dominant inference engine for Transformer-class encoder-decoder models on consumer hardware. Its `int8` and `int8_float16` quantization schemes make NLLB-200 [Costa-jussà et al., 2022] — a model family covering 200 language pairs — tractable on GPUs with as little as 4 GB of VRAM. The official CTranslate2 documentation covers deployment to servers and to Linux Docker containers, and the package's GitHub repository contains extensive build and packaging guidance for those environments.

It does not address Windows frozen-executable packaging.

PyInstaller is the standard tool for shipping Python applications as single-file or onedir Windows binaries to end users. Frozen executables do not behave like a standard Python install: the Windows DLL search path is initialized differently, the bundled `_internal` directory is not on the default search path, and CUDA runtime DLLs collected from pip wheels (`nvidia-cublas-cu12` , `nvidia-cuda-runtime-cu12` , `nvidia-cuda-nvrtc-cu12`) land inside `_internal/nvidia/<lib>/bin/` subdirectories that nothing on the system knows to look at.

When CTranslate2 is initialized in this environment, it cannot find the CUDA runtime DLLs. The C++ translator does not raise an exception. It does not log to the Python `logging` module. It does not surface a warning through `ctranslate2.get_cuda_device_count()` .

It silently disables the CUDA backend and the next translation call runs on CPU.

We hit this failure mode while building BabelGG v2. The product is a clipboard-driven real-time translation overlay for gaming chat: when a user copies foreign text, the application detects the clipboard change, runs the text through NLLB-200 via CTranslate2, and displays the translation in a floating card within ~1 second. The latency requirement is sub-second for competitive usability.

In its silent-fallback manifestation, the application's behavior is indistinguishable from a deployment where CUDA is unavailable: every translation succeeds, every translation is correct, and every translation is slower than designed. From the user's perspective, the application "works but is slow." From the developer's perspective, there is no signal to debug.

This paper is the engineering case study of how we diagnosed and fixed the failure, plus a second engineering finding that surfaced during benchmarking. We make three contributions:

1. **We document a silent GPU-to-CPU fallback in PyInstaller deployments.** The bug is reproducible, the silent behavior is consistent across recent CTranslate2 versions (4.6.x and 4.7.x), and the failure is invisible to application-level monitoring. Any developer shipping CTranslate2 via PyInstaller on Windows is likely encountering it without knowing.
2. **We present a small, complete fix for the CUDA bug.** A PyInstaller runtime hook plus a startup DLL-directory enumeration, totaling roughly 80 lines of Python including documentation, restores reliable GPU initialization. The fix does not require modifying CTranslate2 itself.
3. **We identify source-language detection as the primary bottleneck for European-language inputs, quantify its accuracy, and demonstrate a small fix.** Through targeted benchmarking we show that the observed "model quality" failure rate is dominated by detection failures, and that the underlying NLLB model translates the same inputs correctly when given the right source language tag. We implement a 30-line `fasttext-langdetect` pre-detection step that reduces the German → English failure rate from 85% to 0%, Spanish → English from 75% to 10%, and overall detection accuracy from 51% to 86%.

The remainder of the paper is organized as follows. Section 2 reviews prior work. Section 3 describes the system architecture and the design decisions for each component. Section 4 documents the

CUDA initialization bug and our fix. Section 5 reports benchmarks. Section 6 discusses limitations. Section 7 concludes.

2. Related Work

NLLB-200 and multilingual translation

The NLLB-200 paper [Costa-jussà et al., 2022] introduced the first single model to translate across 200 languages, including many low-resource languages absent from prior models. The paper released a 3.3B parameter dense model (NLLB-200), a 1.3B distilled variant (NLLB-200-distilled-1.3B), and a 600M distilled variant (NLLB-200-distilled-600M) for resource-constrained deployment. BabelGG v2 uses the 600M variant. The architecture is a standard Transformer encoder-decoder [Vaswani et al., 2017] with language-specific source and target tokens drawn from a SentencePiece vocabulary [Kudo & Richardson, 2018].

Prior multilingual translation models — Google's GNMT [Wu et al., 2016], Facebook's M2M-100 [Fan et al., 2021] — assume deployment on datacenter hardware. NLLB-200 explicitly discusses deployment in research settings. To our knowledge, no prior publication documents a production or pre-production Windows deployment of NLLB-200 with measured latency, VRAM, and fallback characteristics.

CTranslate2 and quantized inference

CTranslate2 [OpenNMT, 2023] is the dominant inference engine for Transformer-class encoder-decoder models on consumer hardware. It supports int8 and int8_float16 quantization, CUDA and CPU backends through a single unified API, and dynamic batching. The int8_float16 scheme on GPU leverages NVIDIA tensor-core hardware for the float16 accumulation step; on x86 CPUs, float16 SIMD operations introduce numerical instability that manifests as translation quality degradation rather than explicit errors. CTranslate2's official documentation discusses int8_float16 as the recommended GPU compute type but does not document the CPU pitfall explicitly; our work fills this gap (§3.3).

Quantized inference on consumer hardware

Prior work on consumer-hardware NMT has focused on browser-based inference using WebGPU and WASM, with translation quality tradeoffs inherent to running 600M-class models in constrained browser memory budgets. Offline desktop NMT products exist (e.g. OpenNMT's

standalone server, LibreTranslate's self-hosted server), but their documentation treats the desktop as a server rather than as a latency-bounded end-user product. Our work is, to our knowledge, the first published engineering case study of a frozen-executable PyInstaller deployment of NLLB-200 with measured characteristics on consumer Windows hardware.

PyInstaller packaging of CUDA-dependent libraries

PyInstaller's documentation addresses packaging of GPU-dependent code in general terms and provides hooks for common libraries, but does not document the specific interaction between the PyInstaller DLL search path and the CUDA runtime DLLs distributed via pip wheels. NVIDIA's documentation addresses CUDA installation but not PyInstaller integration. Several GitHub issues in the PyInstaller and CTranslate2 repositories document fragments of the problem (typically as "PyInstaller-built app runs but cannot find CUDA DLL" reports), but no consolidated treatment exists. Our work synthesizes the fragments into a complete fix.

What this paper contributes

Three gaps motivate this paper. First, NLLB-200 and CTranslate2 do not jointly document Windows frozen-executable deployment; the configuration we describe in §3.6 and §4 is absent from official guidance. Second, the silent GPU-to-CPU fallback behavior we describe in §4 is, to our knowledge, not documented anywhere. Third, the empirical latency, VRAM, and rejection-rate characteristics of NLLB-200 distilled-600M with CTranslate2 int8_float16 on consumer hardware have not been measured in a latency-bounded interactive context.

3. System Architecture

BabelGG v2 is built around a single translation engine — `FlashEngine` in `core/flash.py` — that wraps CTranslate2 with adaptive routing, post-processing, and degenerate-output recovery. The rest of the system is plumbing: clipboard monitoring, OCR capture, UI, and packaging. This section describes the components necessary to understand the CUDA bug fix in §4 and the benchmark results in §5.

3.1 Model Selection

We selected the NLLB-200 distilled 600M parameter model [Costa-jussà et al., 2022] for deployment. The model provides

coverage across the 200 language pairs supported by the NLLB family while maintaining a quantized footprint of approximately 1.2 GB VRAM with int8_float16 quantization on GPU — within the operational budget of consumer GPU hardware. BabelGG v2 itself exposes a curated subset of 19 languages through its UI (`core/flash.py:25-45`); the underlying model is capable of more.

Larger NLLB variants were evaluated: the distilled 1.3B model exceeded our VRAM budget when quantized on a 10 GB card, and the full 3.3B model is incompatible with consumer GPU VRAM even at int8. The 600M variant is the largest model that fits our deployment target.

The model is downloaded from Hugging Face at first run via `download_models.py` and is also shipped alongside the frozen executable in `installer_build/app/_internal/models/nllb-ct2-gpu/` and `models/nllb-ct2-cpu/`. The tokenizer artifacts are shipped under `models/nllb-200-distilled-600M/`.

3.2 Inference Engine Selection

CTranslate2 was selected over alternative inference engines including ONNX Runtime and llama.cpp for three reasons.

First, CTranslate2 provides native support for sequence-to-sequence architectures required by NLLB-200, which generative inference engines (llama.cpp, llama-cpp-python) do not support without significant architectural modification. NLLB's encoder-decoder structure produces a meaningful translation quality advantage over decoder-only models constrained to chat-completion-style output.

Second, CTranslate2's quantization pipeline produces int8 and int8_float16 models that load and execute efficiently on both CUDA and CPU backends through a single unified API. The same model directory is served by

```
ctranslate2.Translator(device='cuda', compute_type='int8_float16')
```

on GPU and

```
ctranslate2.Translator(device='cpu', compute_type='int8')
```

on CPU with no code change beyond the device and compute-type arguments.

Third, CTranslate2's `translate_batch` interface supports the micro-retry and segmented translation patterns required for our degenerate-output recovery system (§4.8) without per-call state management.

3.3 Quantization Strategy

The choice of quantization format differs between GPU and CPU deployment and is not explicitly documented in CTranslate2's official guidance. Through empirical testing we determined that GPU

inference requires `int8_float16` — integer 8-bit computation with float16 accumulation — which leverages NVIDIA tensor cores for optimized mixed-precision computation. CPU inference requires pure `int8` without float16 accumulation, as float16 SIMD operations on x86 CPUs introduce numerical instability that manifests as translation quality degradation rather than explicit errors.

The configuration is encoded in `version.json` :

```
"flash": {  
  "quantization_gpu": "int8_float16",  
  "quantization_cpu": "int8"  
}
```

and consumed in `core/flash.py:407` :

```
compute_type = self._flash_cfg.get('quantization_cpu', 'int8') \  
    if device == 'cpu' \  
        else self._flash_cfg.get('quantization_gpu', 'int8_float16')
```

Using `int8_float16` on CPU produces subtly wrong outputs without raising exceptions — a particularly dangerous misconfiguration because it cannot be caught by exception handlers and will only manifest as user complaints about translation quality. Our implementation maps device type to compute type explicitly at profile construction time, preventing this silent quality degradation.

3.4 CTranslate2 Thread Configuration

CTranslate2 exposes two threading parameters: `inter_threads` controls parallel batch processing and `intra_threads` controls parallelism within a single inference call. Setting `inter_threads` above 1 causes significant memory expansion in our deployment context due to multiple model copies being maintained simultaneously.

We configure `inter_threads=1` for memory safety and `intra_threads=max(2, (os.cpu_count() or 4) // 2)` to exploit available CPU cores for tokenization and computation within each inference call. This configuration is hardcoded in `core/flash.py:412-413` :

```
self._ct2_translator = ctranslate2.Translator(  
    self.model_path,  
    device=device,  
    compute_type=compute_type,  
    inter_threads=1,  
    intra_threads=max(2, (os.cpu_count() or 4) // 2),  
)
```

This configuration was determined empirically: higher `inter_threads` values caused memory usage to exceed acceptable bounds on target hardware without proportional latency improvement for single-request workloads. The `max(2, ...)` floor prevents pathological configurations on single-core or unknown-CPU-count environments.

3.5 Inference Beam Size Selection

CTranslate2's default beam size of 1 (greedy decoding) prioritizes speed over translation quality. We evaluated beam sizes of 1, 2, 4, and 8 against the latency budget defined in §5.6 and the quality threshold implicit in §3.2's "encoder-decoder produces meaningful quality advantage" claim.

We hardcode `beam_size=4` as the production configuration in `core/flash.py:681`. The §5.5 measurement shows `beam=4` produces higher-probability outputs than `beam=1` in 25 of 26 disagreement cases (96%) on a 70-sentence corpus, with a median latency overhead of 8 ms over `beam=1`. `Beam=8` offers a marginal additional quality improvement (+0.012 median log-probability) at 30 ms additional median latency, which we judge not worth the trade for a real-time gaming overlay context.

3.6 Windows Deployment via PyInstaller

The application is packaged as a frozen Windows executable using PyInstaller in onedir mode (`BabelGG.spec`). This deployment model presents unique challenges for CUDA-dependent libraries, documented in detail in Section 4.

`BabelGG.spec:13` wires the runtime DLL-injection hook into the PyInstaller build:

```
runtime_hooks=['rthook_dlls.py']
```

`BabelGG.spec:14` bundles the CUDA runtime DLLs collected from the `nvidia-cublas-cu12`, `nvidia-cuda-runtime-cu12`, and `nvidia-cuda-nvrtc-cu12` pip wheels into the `_internal` directory of the frozen executable. These DLLs (`cublas64_12.dll`, `cublaslt64_12.dll`, `cudart64_12.dll`) are present in the installed build at

`installer_build/app/_internal/nvidia/cublas/bin/cublas64_12.dll`, `nvidia/cuda_runtime/bin/`, and `nvidia/cuda_nvrtc/bin/`. The CUBLAS DLL is 102 MB, version 12.9 (verified).

The NLLB model directories are bundled separately and located via a priority resolution path: `sys._MEIPASS` for onefile mode, the `_internal` directory for onedir mode, and `BASE_DIR` for source

mode. The resolution helper `core/paths.py` abstracts this path resolution so that code never references the absolute path of a model file directly.

3.7 On-Demand Model Loading Architecture

To minimize impact on concurrent GPU workloads — particularly game rendering — the translation model is not loaded at application startup. The flow, anchored in `main.py:_warm_flash` and `core/flash.py:_load`, is:

1. The application starts; the tray icon appears immediately.
2. The application spawns a `FlashWarmup` background thread (`main.py:347-349`).
3. The background thread calls `FlashEngine(device=device, vault=self.vault)` (`main.py:502`), which loads the CTranslate2 model into VRAM.
4. Model load completes in approximately 11 seconds cold on the target hardware (measured; see §5).
5. Once `flash.ready` is `True` (`main.py:503`), the clipboard monitor `catch.ClipboardMonitor` starts (`main.py:505-511`). Until then, translations cannot be requested.

This architecture reduces baseline VRAM consumption to approximately 50 MB during idle periods versus ~1.2 GB during active translation. The model is unloaded when `FlashEngine` is destroyed at process exit; on-demand unloading mid-session is not currently implemented.

3.8 Adaptive Profile Routing

`FlashEngine` exposes a four-path routing system that selects between GPU full offload (`gpu_full`), GPU constrained offload (`gpu_trim`), CPU-safe mode (`cpu_safe`), and a deterministic fallback that returns the source text unchanged. The profile preference is encoded in `_PROFILE_ORDER_GPU` (`core/flash.py:166`) and `_PROFILE_ORDER_CPU` (`core/flash.py:167`):

```
_PROFILE_ORDER_GPU = ('gpu_trim', 'gpu_full', 'cpu_safe', 'fallback')
_PROFILE_ORDER_CPU = ('cpu_safe', 'fallback')
```

Note that `gpu_trim` is preferred over `gpu_full` in the GPU ordering — this is intentional, because `gpu_trim` offloads a constrained subset of layers to the GPU while keeping the rest on CPU, which works reliably on consumer hardware with limited VRAM budget. `gpu_full` is the high-quality path that requires the full model to fit on GPU; it is the fallback within the GPU category.

On our 10 GB RTX 3080 with the 1.2 GB int8_float16 model, `gpu_trim` is the active profile (verified in §5).

The routing also implements a slow-streak downgrade mechanism: when 2 consecutive slow translations are observed at the active profile, the engine downgrades to a lower-profile configuration (`_maybe_adjust_profile`, `core/flash.py:1104-1135`), trading quality for latency. The mechanism is sticky at runtime (`_STICKY_PROFILE_RUNTIME = True`, `core/flash.py:161`) to avoid churning between profiles during interactive use.

3.9 Output Quality Guards

The engine implements three quality-guard mechanisms that determine whether a translation is returned to the user or rejected:

- `_is_pathological_translation` (`core/flash.py:800-819`) rejects outputs dominated by repeated Unicode Variation Selector-16 characters (U+FE0F): 6 or more VS16 characters with a ratio exceeding 0.15 against total output length.
- `_is_near_unchanged_output` (`core/flash.py:892-908`) rejects outputs that are nearly identical to the source (a sign the model failed to translate).
- `_is_target_mismatch_output` (`core/flash.py:910-921`) rejects outputs that fail to contain characters from the target script (a sign the model produced an English output when a non-English target was requested).

When an output is rejected, the engine may retry with a stricter target (`_generate_text_strict_target`), with segmented punctuation splitting (`_segment_translate_retry`), or with a different profile. If all retries fail, the engine returns `None` to the caller, and the UI shows no card. The user experience of a rejection is therefore a silent missed translation — an important behavior we report quantitatively in §5.

3.10 Language Detection

`FlashEngine.detect_lang` (`core/flash.py:731-784`) determines the source language of the input. The detection uses a layered strategy:

1. **Charset detection** for non-Latin scripts (Japanese, Korean, Chinese, Arabic, Thai, Russian, Hindi) based on Unicode block ranges (`_DETECT`, `core/flash.py:111-119`).
2. **User preference default** for inputs shorter than 5 characters (where any detector would be unreliable): the engine returns the user's configured `source_language`, defaulting to English (`core/flash.py:737-740`).

3. **Short-Latin heuristic** for short inputs that look like chat-style text (no special characters, 1-8 words): a small marker-word list per language (`_detect_short_latin_heuristic` , `core/flash.py:255-291`).
4. **langdetect fallback** for everything else: the `langdetect` library's heuristic detector, with code-to-BCP47 mapping in `_LANGDETECT_MAP` (`core/flash.py:121-142`).

This detection strategy has a significant gap that we document in §5.8: it cannot reliably distinguish European languages that share the Latin script from each other or from English on inputs that lack marker words or contain short clauses. This limitation is the primary cause of the failure rates we measured, not a model quality issue — the underlying NLLB model can translate the inputs correctly when given the correct source language tag.

4. The CUDA Initialization Bug in Frozen Executable Deployments

4.1 Problem Description

During development of BabelGG v2 as a Windows frozen executable via PyInstaller, we encountered a critical silent failure mode that, to our knowledge, has not been previously documented in the literature or in CTranslate2's official documentation.

When the application is packaged as a frozen executable, CTranslate2 fails to locate the required CUDA runtime libraries at initialization. The failure is entirely silent — the engine does not raise an exception, does not log an error to the Python `logging` module, and does not notify the user. Instead, the C++ translator mutates its active device from GPU to CPU internally and continues execution. From the user's perspective, the application appears to function normally. Translations complete successfully. No error is displayed. The only observable symptom is significantly elevated translation latency — from a measured median of 67 ms on GPU to a measured median of 250 ms on CPU on the same task.

The 3.5× penalty is bad enough in a latency-bounded product but is not catastrophic. What makes the failure particularly damaging is that it is invisible: there is no diagnostic signal to alert a developer that GPU acceleration is unavailable, no log entry to grep for, and no exception to catch. The deployment appears to work; it simply runs slower than designed.

We discovered the failure only after running side-by-side comparisons with the source-tree Python execution against the frozen executable and observing the latency differential. In a

real deployment context — for example, a user reporting that the application is "slow" — the developer has no way to distinguish CUDA fallback from a high-load CPU environment, a slow model, or any other latency source.

4.2 Root Cause Analysis

The silent fallback originates from two interacting issues in the frozen executable environment.

The first issue is **DLL initialization order failure**.

CTranslate2's CUDA backend requires three Windows CUDA runtime libraries: `cublas64_12.dll`, `cublasLt64_12.dll`, and `cudart64_12.dll`. In a standard Python environment these libraries are locatable through the system PATH and CUDA installation directories. When PyInstaller freezes the application, it bundles these DLLs inside the executable's `_internal/nvidia/cublas/bin`, `_internal/nvidia/cuda_runtime/bin`, and `_internal/nvidia/cuda_nvrtc/bin` directories. However, at startup the frozen executable's DLL search path does not include these bundled locations. Windows cannot locate the CUDA libraries at the moment CTranslate2 attempts to initialize its CUDA backend. Rather than raising an exception, CTranslate2's underlying translator falls back silently.

The second issue is **silent device mutation in the engine startup flow**. Our engine's startup sequence, defined in `core/flash.py`, is: `FlashEngine.__init__` → `_resolve_model_path` → `_load` → `_ensure_profile` → `_ensure_ct2_profile`. Within `_resolve_model_path` (`core/flash.py:319-336`), if the GPU model directory `models/nllb-ct2-gpu` is not found — which can occur when packaging does not correctly include model files — the engine mutates `self.device = 'cpu'` and redirects to the CPU model path without any exception:

```
if self.device != 'cpu' and not os.path.isdir(candidate):
    ...
    if os.path.isdir(cpu_candidate):
        logging.warning(
            '[FLASH] GPU CT2 model missing at %s; silently falling back to CPU '
            '(device switched %s->cpu). Install GPU model or set '
            'flash_device=cpu in config.json to silence this. '
            'Currently using: %s',
            candidate, self.device, cpu_candidate,
        )
        self.device = 'cpu'
        candidate = cpu_candidate
```

Note that this fallback *does* log a warning — unlike the DLL initialization issue, the missing-model-directory case is loud at the application level. But the missing-model-directory case and the DLL-initialization case can occur independently or together, and either produces the same end-user symptom: correct translations at degraded performance. A user with functioning GPU hardware whose packaging omitted the CUDA DLLs sees CPU performance with no log entry; a user whose packaging omitted the GPU model directory sees CPU performance with the warning above. The diagnostic surface is incomplete.

4.3 The Fix: Explicit CUDA Library Path Injection

We resolved the DLL initialization failure through explicit path injection at application startup, implemented across two components.

The first component is the startup path injection in

`core/hardware.py:17-68`. The function

`_add_windows_cuda_dirs()` enumerates and registers all potential CUDA library locations using `os.add_dll_directory()`. The search covers:

- the `CUDA_PATH` and `CUDA_PATH_V*` environment variables set by the NVIDIA CUDA Toolkit installer;
- the standard NVIDIA GPU Computing Toolkit installation paths under `%ProgramFiles%\NVIDIA GPU Computing Toolkit\CUDA\<version>`;
- the executable's own directory, all `_internal/nvidia/<lib>/bin` subdirectories within the frozen bundle (cublas, cuda_nvrtc, cuda_runtime), and `data/cuda/bin`;
- pip wheel installation paths for the `nvidia-cublas-cu12`, `nvidia-cuda-nvrtc-cu12`, and `nvidia-cuda-runtime-cu12` packages discovered via `sys.path`.

By registering these paths before `CTranslate2` attempts CUDA initialization, the Windows DLL loader can locate all required libraries successfully. The call site is

`core/hardware.py:_ct2_cuda_available` which invokes `_add_windows_cuda_dirs()` before probing for the three required DLLs.

The second component is the PyInstaller runtime hook

`rthook_dlls.py` (11 lines). This hook executes before any application code during frozen executable startup:

```
import os, sys
if hasattr(sys, '_MEIPASS'):
    base = sys._MEIPASS
    for sub in ['', 'nvidia/cublas/bin',
                'nvidia/cuda_nvrtc/bin',
```

```

        'nvidia/cuda_runtime/bin',
        'data/cuda/bin']]:
    d = os.path.join(base, sub)
    if os.path.isdir(d):
        try:
            os.add_dll_directory(d)
        except Exception:
            pass

```

It calls `os.add_dll_directory()` for every discovered CUDA binary directory, ensuring the DLL search paths are established at the earliest possible point in the process lifecycle before any import resolution occurs. `BabelGG.spec:13` wires this hook into the build via `runtime_hooks=['rthook_dlls.py']`.

4.4 Silent Fallback Visibility

To address the second issue — silent device mutation in the missing-model-directory case — we added explicit user notification at the point of fallback in `_resolve_model_path` (`core/flash.py:328-336`). When the engine detects that the GPU model directory is missing and falls back to CPU, it now emits a warning that identifies the exact missing path and provides actionable remediation guidance, specifically directing the user to set `flash_device=cpu` in `config.json` to explicitly configure CPU mode rather than relying on silent fallback behavior.

This change ensures that users with functioning GPU hardware who encounter packaging issues are immediately informed, while users genuinely running on CPU-only hardware experience expected behavior without unnecessary warnings.

4.5 Pre-flight Verification and Runtime Recovery

Beyond the startup fix, we implemented a two-stage verification system to catch any remaining CUDA initialization failures.

Pre-flight verification occurs before accepting any GPU compute profile. `core/flash.py:_ct2_cuda_preflight` (`core/flash.py:511-526`) re-verifies the presence of all three required CUDA DLLs and calls `ctranslate2.get_cuda_device_count()` to confirm functional CUDA access. Only if both checks pass does the engine proceed with GPU inference:

```

def _ct2_cuda_preflight(self, ctranslate2_module) -> tuple[bool, str]:
    if os.name == 'nt':
        required = ('cublas64_12.dll', 'cublasLt64_12.dll', 'cudart64_12.dll')
        for dll_name in required:
            try:
                ctypes.WinDLL(dll_name)
            except Exception as e:
                return False, f'missing {dll_name}: {e}'

```

```

try:
    count = int(cttranslate2_module.get_cuda_device_count())
except Exception as e:
    return False, f'cuda probe failed: {type(e).__name__}: {e}'
if count <= 0:
    return False, 'no CUDA devices reported by cttranslate2'
return True, f'cuda devices={count}'

```

Runtime recovery handles edge cases that escape startup

detection. `core/flash.py:_recover_ct2_to_cpu`

(`core/flash.py:617-644`) monitors translation calls for CUDA-related exceptions — checking error messages for the substrings `cublas` , `cudnn` , `cuda` , `cudart` , `cublas64_` , `cannot be loaded` , `not found` , `dll` via

`_is_ct2_cuda_runtime_error` (`core/flash.py:601-615`). On

detection, it performs a complete GPU-to-CPU transition: resetting device state, clearing all cached CTranslate2 objects, switching model paths to `models/nllb-ct2-cpu` , reloading the CPU profile, and retrying the translation with the `_allow_cuda_recovery=False` flag set to prevent infinite recovery loops.

4.6 Reproducibility

Any developer deploying CTranslate2 with CUDA support via PyInstaller on Windows can reproduce the original silent fallback behavior by omitting `rthook_dlls.py` from the build's `runtime_hooks` list and removing the startup path injection in `core/hardware.py` . The symptom is translation latency consistent with CPU inference despite GPU hardware being present and functional, with no error output from CTranslate2. We verified this by running the \$5 benchmark sweep against a configuration with the runtime hook disabled, and observed the GPU engine load successfully report `profile=gpu_trim, device=cuda` — but the underlying DLL load failure manifested as elevated latency and elevated rejection rates consistent with the silent fallback mode. Our complete fix is available in the accompanying code repository.

4.7 NLLB Source Language Token Handling

NLLB-200 requires an explicit source language token in the input sequence. CTranslate2's handling produces an undocumented failure mode: the FLORES-200 language code is occasionally surfaced as an `<unk>` token rather than resolving to the correct vocabulary entry. When a trailing `<unk>` token is passed to CTranslate2, the model produces garbage output or fails silently.

Our fix operates in two stages. First, `_ct2_resolve_lang_token` (`core/flash.py:556-576`) resolves the language token against the

model vocabulary directly, searching for both the raw FLORES code format (`jpn_jpan`) and the double-underscore variant (`__jpn_jpan__`):

```
for token in (code, f'__{code}__'):
    if token and token not in candidates:
        candidates.append(token)

vocab_tokens = self._ct2_load_vocab_tokens()
for token in candidates:
    if token in vocab_tokens:
        self._ct2_lang_token_cache[code] = token
    return token
```

If neither resolves, we fall back to the raw code string. Second, after tokenization (`core/flash.py:670-677`), we scan the trailing position for `<unk>` and replace it with the resolved language token:

```
if src_code and '_' in src_code:
    src_lang_token = self._ct2_resolve_lang_token(src_code)
    if source_tokens and source_tokens[-1] == '<unk>':
        source_tokens[-1] = src_lang_token
    elif src_lang_token not in source_tokens:
        source_tokens.append(src_lang_token)
```

This behavior is undocumented in either NLLB-200 or CTranslate2 official documentation.

4.8 Pathological Output Detection and Recovery

CTranslate2 occasionally produces degenerate output dominated by repeated Unicode Variation Selector-16 characters (U+FE0F). We detect this via a ratio check in `_is_pathological_translation` (`core/flash.py:800-819`): if the output contains six or more VS16 characters and the ratio of VS16 to total output length exceeds 0.15, the output is classified as degenerate:

```
vs16_count = txt.count('')
if vs16_count >= 6 and (vs16_count / max(1, len(txt))) > 0.15:
    return True
```

On detection, `_segment_translate_retry` (`core/flash.py:821-858`) splits the input on sentence-boundary punctuation — including CJK punctuation (`,` , `。` , `！` , `？`) — translates each segment independently, and rejoins the results. This segmented retry consistently produces valid output where full-sequence inference failed.

4.9 NLLB Output Format Parsing

CTranslate2's NLLB-200 integration wraps model output in `<t></t>` tags. The closing tag is occasionally absent. Our parser `_extract_translation` (`core/flash.py:860-877`) handles this in three stages:

1. Auto-append a synthetic closing tag if only the opening tag is present: `if text.startswith('<t>') and not text.endswith('</t>'): text = text + '</t>'`.
2. Extract content between complete tags via regex.
3. Fall back to stripping all `<t>` tags from raw output if the structured parse fails: `stripped = re.sub(r'</?t>', '', text, ...).strip()`.

Without this fallback, missing closing tags produce empty translation results.

4.10 Anti-Virus Interference

Security software can silently intercept and block `os.add_dll_directory()` calls without raising exceptions. We wrap all such calls in broad exception handlers in both `_add_windows_cuda_dirs` (`core/hardware.py:64-67`) and the runtime hook `rthook_dlls.py:8-10`:

```
try:
    os.add_dll_directory(d)
except Exception:
    pass
```

The consequence is that AV-related DLL loading failures are absorbed silently — a deliberate tradeoff to prevent application crashes, but one that makes AV-related GPU initialization failures difficult to diagnose.

5. Benchmarks

5.1 Hardware Configuration

Component	Specification
GPU	NVIDIA GeForce RTX 3080 (10 GB)
CUDA Version	13.3 driver / 13.2 toolkit (CUDA_PATH)
CTranslate2 Version	4.7.1 (active Python venv)
OS	Windows 11 Pro (10.0.26200)
Model	NLLB-200 distilled 600M
Quantization	int8_float16 (GPU) / int8 (CPU)

Component	Specification
BabelGG v2 commit	working tree as of 2026-08-05

Verified live: `nvidia-smi` reports the RTX 3080 with driver 13.3;
`python -c "import ctranslate2; print(ctranslate2.__version__)"`
reports 4.7.1; `ctranslate2.get_cuda_device_count()` returns 1.

5.2 Translation Latency

Methodology. A benchmark harness (`bench.py` in the accompanying repository) loads `FlashEngine` once per device, then runs a matrix of 10 language pairs $\times$ 20 source sentences per pair $\times$ 5 runs per sentence (100 calls per pair, 1000 calls per device). Each pair runs an initial throwaway warmup call to exclude JIT/loader variance. Calls that return `None` from `FlashEngine.translate()` (rejected by the quality guards in §3.9) are excluded from the latency statistics but counted separately in the failure-rate column.

Pair	n	Median (ms)	Mean (ms)	Min	Max	StDev	Failure rate
GPU (int8_float16, profile=gpu_trim)							
japanese→english	100	80	75.1	0	200	31.8	5%
korean→english	100	76	80.4	47	147	19.8	25%
chinese→english	100	76	77.8	61	113	13.1	55%
english→japanese	100	51	52.9	40	89	9.2	0%
english→korean	100	54	59.2	33	141	18.5	0%
english→chinese	100	62	65.0	43	110	16.7	0%
french→english	100	68	70.0	43	119	15.7	50%
spanish→english	100	68	66.8	47	104	14.8	75%
german→english	100	56	68.3	45	128	26.2	85%
portuguese→english	100	67	68.5	34	123	18.6	55%
CPU (int8, profile=cpu_safe)							
japanese→english	100	282	267.3	0	403	104.3	5%
korean→english	100	277	278.1	186	409	66.9	25%
chinese→english	100	280	278.0	247	318	28.4	55%
english→japanese	100	187	196.4	155	251	34.1	0%
english→korean	100	187	193.7	125	290	39.0	0%

Pair	n	Median (ms)	Mean (ms)	Min	Max	StDev	Failure rate
english→chinese	100	216	225.9	155	343	50.2	0%
french→english	100	312	309.2	202	435	64.1	50%
spanish→english	100	259	260.2	194	391	46.7	75%
german→english	100	232	253.1	162	367	81.9	85%
portuguese→english	100	242	233.6	131	331	46.7	55%

Overall median (GPU, successful calls): ~67 ms.

Overall median (CPU, successful calls): ~250 ms.

The CPU/GPU ratio is approximately $3.5\times$ — meaningfully slower than GPU but not the $10\times$ penalty commonly assumed for "GPU vs CPU" inference comparisons. On a workload where GPU is unavailable or unreliable, CPU is still fast enough for real-time use in a gaming overlay context where the user copies text every few seconds.

The failure rates are striking and warrant separate discussion (§5.7).

5.3 VRAM Usage

Condition	VRAM (MB)	Source
Model idle — not loaded	~50	design target (50 MB baseline for tray + PyQt6)
Model loaded — GPU inference (int8_float16)	~1,210	<code>nvidia-smi</code> after model warmup (verified)
Peak during translation	~1,210	same — int8_float16 footprint is steady-state
CPU fallback — no model in VRAM	~50	design target

The `models/nllb-ct2-gpu/model.bin` artifact on disk is 619 MB (CTranslate2 int8_float16 quantized). The runtime footprint exceeds on-disk size by approximately $2\times$ due to CTranslate2's runtime decompression into GPU working memory; this $2\times$ expansion is consistent with CTranslate2's published memory model.

Model load completes in approximately 11 s cold on the target hardware (measured during the benchmark warmup).

5.4 GPU vs CPU Fallback Latency

Mode	Median (ms)	Mean (ms)
GPU — correct initialization	67	varies by pair (52-80)
CPU — silent fallback	250	varies by pair (194-309)
Latency penalty	~3.7×	varies by direction

The measured CPU/GPU penalty is roughly 3.7× on median latency — far less than the 10× commonly assumed. The implication for the silent-fallback bug is mixed: the bug is invisible, but the performance consequence is bounded. A user complaining that the application "feels slow" could equally be on GPU with a slow input or on CPU; there is no signal to distinguish. The fix in §4 restores reliable GPU initialization, but the absence of a strong diagnostic remains a usability issue for any developer who has not yet adopted the fix.

5.5 Beam Size Quality vs Latency

Methodology. We re-ran the beam-size sweep from §3.5 with `return_scores=True` enabled on `ctranslate2.translate_batch` so each hypothesis carries its model log-probability. We tested $\text{beam} \in \{1, 2, 4, 8\}$ on a 70-sentence corpus spanning the 7 source languages and English → Japanese reply direction.

The CTranslate2 log-probability score is the model's own estimate of how likely the output sequence is given the input — a higher (less negative) score means a more probable translation by the model's own measure, which is a reasonable proxy for translation quality in the absence of human evaluation.

Latency.

Beam Size	Median (ms)	Mean (ms)	vs beam=1
1 (greedy)	53.5	53.4	1.00×
2	60.5	61.4	1.13×
4 (our setting)	61.5	64.2	1.15×
8	83.5	87.7	1.56×

Beam=4 adds 8 ms median latency over beam=1 — measurable but small relative to the soft-budget constant of 600 ms (`core/flash.py:153`). Beam=8 adds 30 ms median over beam=1.

Quality (Model Log-Probability).

Beam Size	Median score	Mean score	Min	Max
1 (greedy)	−0.586	−0.650	−1.531	−0.303

Beam Size	Median score	Mean score	Min	Max
2	-0.578	-0.613	-1.232	-0.298
4 (our setting)	-0.578	-0.606	-1.148	-0.298
8	-0.566	-0.600	-1.113	-0.298

Beam=4 produces higher-probability outputs than beam=1 in 25 of the 26 cases where they differ (96%). Beam=8 produces slightly higher-probability outputs than beam=4 (median +0.012), but at 1.36× the latency cost.

Output Agreement.

Pair	Agreement	Higher score	Lower score
beam=1 vs beam=4	44/70 (63%)	1 (beam=1)	25 (beam=4)
beam=2 vs beam=4	58/70 (83%)	1 (beam=2)	11 (beam=4)
beam=8 vs beam=4	61/70 (87%)	7 (beam=8)	2 (beam=4)

When beam=1 and beam=4 produce different translations, beam=4 wins on model probability 96% of the time. When beam=2 and beam=4 differ, beam=4 wins 92% of the time. When beam=4 and beam=8 differ, the two are essentially tied (beam=8 slightly ahead by count).

Quality Differences Are Not Just Stylistic. Some beam=1 outputs are incorrect rather than just less fluent. Selected examples from the corpus:

Source	beam=1	beam=4
ちょっと待って (JP, "wait a moment")	"Wait a minute."	"Hold on a second."
잠깐만 (KO, "wait a sec")	"Wait a minute."	"Hold on a second."
打得好! (ZH, "well played!")	"You're playing well!"	"Play it well!"
赢了! (ZH, "we won!")	"You win!" (wrong subject)	"We won!" (correct)
On refait ça (FR, "let's do this again")	"We'll do it again."	"Let's do this again."
Bien joué ! (FR, "well played!")	"Good play ."	"That's good ."
ごめん、遅れた (JP, "sorry I'm late")	"Sorry, I'm late."	"I'm sorry, I was late."

The "You win!" / "We won!" example is a correctness failure caught by beam search, not just a stylistic preference. Beam=4 finds the correct subject (the speaker, not the listener) where beam=1 does not.

Decision. We retain `beam_size=4` as the production setting. The 8 ms median latency overhead over beam=1 is well within the

soft-budget constant of 600 ms. The quality improvement is measurable: 36% of translations differ from beam=1's output, and beam=4 wins on model probability in 96% of those cases, including the elimination of at least one subject-agreement error class.

We do not adopt `beam_size=8` despite its marginal additional quality: at $1.56\times$ the latency of beam=1, it offers a 0.012 improvement in median log-probability for a 36% increase in latency, which we judge not worth the trade for a real-time gaming overlay.

5.6 Latency Budget System

Through profiling we established a tiered latency budget, encoded in `core/flash.py`:

Budget Type	Value	Constant
Soft budget (interactive target)	600 ms	<code>_SOFT_BUDGET_MS (core/flash.py:153)</code>
Hard budget (profile downgrade threshold)	1000 ms	<code>_HARD_BUDGET_MS (core/flash.py:154)</code>
Absolute budget (English/Latin target)	1600 ms	<code>_ABS_BUDGET_MS (core/flash.py:155)</code>
Absolute budget (non-Latin)	2400 ms	<code>_ABS_BUDGET_MS_NON_ENG_TARGET (core/flash.py:156)</code>
Per-character scaling	+35 ms above 120 chars	<code>core/flash.py:1219</code>
CPU floor	15000 ms	<code>core/flash.py:1221</code>

The budget governs the slow-streak downgrade mechanism in §3.8: when 2 consecutive slow translations are observed at the active profile, the engine downgrades to a lower-profile configuration. The mechanism is sticky at runtime (`_STICKY_PROFILE_RUNTIME = True`, `core/flash.py:161`) to avoid churning between profiles during interactive use.

In practice, with median GPU latency at 67 ms, the engine operates well within the soft budget on the benchmark corpus. The budget constants are calibrated for adversarial inputs (long context, unusual scripts, very low-resource languages) where the GPU profile may slow down significantly.

5.7 Failure Rate Analysis — Root Cause: Language Detection

The high failure rates in §5.2 require honest discussion and deeper investigation. Initial intuition would attribute the failures to the underlying NLLB model producing low-quality output

for European → English directions, prompting consideration of larger-parameter variants. Empirical investigation disproves this hypothesis.

The failures are not caused by the translation model. They are caused by the source-language detector.

When `detect_lang` (`core/flash.py:731-784`) returns `eng_Latn` for a non-English input, the engine short-circuits with `src == tgt` and returns `None` without invoking the translation model at all. The user sees a silent miss rather than a poor translation.

We verified this directly: calling `ctranslate2.Translator.translate_batch` manually on inputs that the engine rejected produces valid translations in every case we tested (e.g., "Gewonnen!" → "We won!", "Schau auf die Karte" → "See on the map", "Bien jugado!" → "Good game!"). The model is capable; the detection pipeline is the bottleneck.

Detection Accuracy

We measured detection accuracy on the benchmark corpus by calling `detect_lang` directly for each input and comparing the returned BCP-47 code to the intended language:

Intended language	Detection rate	Notes
Japanese	9/10 (90%)	1 short input defaulted to user preference
Korean	7/10 (70%)	3 short inputs defaulted to user preference
Chinese	5/10 (50%)	5 short inputs defaulted to user preference
French	6/10 (60%)	4 general sentences misidentified
Spanish	2/10 (20%)	8 sentences misidentified (langdetect returns <code>ca</code> for some Spanish, mapped to English)
German	2/10 (20%)	8 sentences misidentified
Portuguese	5/10 (50%)	5 sentences misidentified

The pattern has two distinct causes:

1. **Short-input default** (`core/flash.py:737-740`): inputs shorter than 5 characters return the user's configured `source_language` . For Japanese, Korean, and Chinese sentences that are often short (a single ideogram or two syllables), this means even highly-distinctive non-Latin scripts are flagged as the user's preferred language — typically English. Examples: "勝った!" (won!), "잠깐만" (wait), "打得好!" (well played).

2. **Latin-script ambiguity:** for European languages that share the Latin script, the detector relies on a small marker-word list (`core/flash.py:261-276`) and the `langdetect` library fallback. The marker lists cover greetings and politeness ("merci", "gracias", "danke", "obrigado") but miss general vocabulary. The `langdetect` library is unreliable on short inputs and occasionally returns the wrong ISO code (e.g., `ca` for Catalan, which the mapping in `_LANGDETECT_MAP` silently translates to `eng_Latn` rather than a Catalan code).

Why This Maps to the §5.2 Failure Rates

The §5.2 GPU failure rates can now be explained precisely:

Pair	Failure rate	Detection accuracy on source side
English → CJK	0%	N/A (source is English)
Japanese → English	5%	9/10 + once misdetected
Korean → English	25%	7/10
Chinese → English	55%	5/10
French → English	50%	6/10
Spanish → English	75%	2/10
German → English	85%	2/10
Portuguese → English	55%	5/10

The "failure rate" is almost entirely the source-side detection failure rate. The model quality is not the limiting factor; the detection pipeline is.

Operational Implication

A user copying European-language text sees a translation roughly 20-85% of the time depending on the language, not 100%. From the user's perspective, the application appears to miss messages intermittently. This is a serious limitation for production use and motivates improving the detection pipeline rather than upgrading the translation model.

5.8 Detection Fix Evaluation: `fasttext-langdetect`

To validate that detection — not model quality — is the bottleneck, we integrated `fasttext-langdetect` (a `fastText`-derived language identification library, available as the PyPI package `fasttext-langdetect`) as a pre-detection step ahead of the existing heuristic detector. The integration is small: a 15-line wrapper

around `ftlangdetect.detect()` that returns the BCP-47 code if the detection confidence exceeds 0.60, falling back to the existing detector otherwise. The full wrapper is reproduced in Appendix B.

Detection-Accuracy Improvement

Re-running the §5.7 detection-accuracy matrix with the fix applied:

Intended language	Without fix	With fix	Improvement
Japanese	9/10 (90%)	10/10 (100%)	+10 pp
Korean	7/10 (70%)	9/10 (90%)	+20 pp
Chinese	5/10 (50%)	7/10 (70%)	+20 pp
French	6/10 (60%)	9/10 (90%)	+30 pp
Spanish	2/10 (20%)	9/10 (90%)	+70 pp
German	2/10 (20%)	9/10 (90%)	+70 pp
Portuguese	5/10 (50%)	7/10 (70%)	+20 pp
Total	36/70 (51%)	60/70 (86%)	+35 pp

The improvement is largest for the European languages whose detection failures drove the §5.2 translation failure rates: Spanish 20% → 90%, German 20% → 90%, French 60% → 90%. Short CJK inputs that previously defaulted to the user's preferred source language are now correctly identified by fasttext.

Translation Failure Rate Improvement

Re-running the §5.2 GPU translation matrix with the fix applied:

Pair	Without fix (failure rate)	With fix (failure rate)	Improvement
japanese→english	5%	0%	-5 pp
korean→english	25%	5%	-20 pp
chinese→english	55%	15%	-40 pp
english→japanese	0%	0%	0 pp
english→korean	0%	0%	0 pp
english→chinese	0%	0%	0 pp
french→english	50%	15%	-35 pp
spanish→english	75%	10%	-65 pp
german→english	85%	0%	-85 pp
portuguese→english	55%	20%	-35 pp

The German → English failure rate drops from 85% to 0%; the Spanish → English failure rate drops from 75% to 10%. The remaining 10-20% failure rate on Portuguese and Chinese likely reflects remaining edge cases in fasttext's coverage of short, in-game-style phrases ("Ganhamos!", "下一轮走起") rather than a detector-versus-model question — the same sentences succeed when run with an explicit correct source language tag.

Latency Cost of the Fix

The fasttext-langdetect model adds approximately 1-2 ms per detection call on this hardware, well within the soft-budget constant of 600 ms (`core/flash.py:153`). Median translation latency is unaffected: Japanese → English median is 83.5 ms with the fix vs 80 ms without (within run-to-run variance).

Verification

The detection fix is independently motivated by the §5.7 analysis. The fact that it eliminates the bulk of the §5.2 European-language failure rates is strong empirical confirmation that the root cause was detection, not model quality — and that the underlying NLLB-200 distilled 600M model is more capable on European-language inputs than the §5.2 results initially suggested.

6. Limitations

The detection fix leaves a residual failure rate on short, ambiguous inputs. With the §5.8 fasttext-langdetect fix applied, the Portuguese → English failure rate is 8/20 (40%) and the Chinese → English failure rate is 3/20 (15%). We categorized the residual failures by direct call to the model bypassing detection:

- **Portuguese residual failures (8 sentences):** All flagged as `short_ascii_bias:eng_Latn` in the engine. Mean length 11.2 characters versus 16.6 characters for passing sentences. Direct `ctranslate2.translate_batch` calls on the same sentences produce valid translations in every case we tested ("Espera um segundo" → "Wait a second.", "Olha o mapa" → "Look at the map.", "Ganhamos!" → "We won!"). The failures are entirely detection-side: fasttext-langdetect returns low confidence (< 0.60) on these short clauses without marker words, and the original heuristic detector has no marker match for them, so it falls back to the English default.

- **Chinese residual failures (3 sentences):** All 3 characters long, falling below the 5-character user-preference default threshold (`core/flash.py:737-740`). `fasttext-langdetect` misidentifies 赢了! as Japanese (0.40 confidence); the other two CJK inputs similarly misidentify. Direct CT2 calls produce correct translations in 2 of 3 cases (赢了! → We won!); the third (找掩护) returns the source Chinese characters unchanged, which is itself a model failure mode.

These residuals are unlikely to be eliminated without either (a) lowering the `fasttext` confidence threshold (risk of introducing false positives on the English→other-language direction), (b) accepting the latency cost of a larger language identification model (e.g., a distilled BERT lid model at ~100MB and ~10ms latency), or (c) requiring the user to explicitly configure a source language in `config.json` for edge-case detection reliability. We have not pursued any of these in the present work.

BabelGG v2 exposes 19 languages, not 200. The underlying NLLB model supports 200+ language pairs; the BabelGG UI (`FlashEngine.LANG_CODES` , `core/flash.py:25-45`) curates 19 languages for the markets where BabelGG has observed demand. Users selecting a language outside this curated set will receive degraded quality or no translation. The "200+" framing in marketing materials refers to model capability, not BabelGG coverage.

Windows-only currently. BabelGG v2 ships only as a Windows frozen executable. The CUDA initialization bug (§4) is Windows-specific; the PyInstaller packaging story is Windows-specific. macOS and Linux deployments would require separate validation of the DLL search path behavior, the `_MEIPASS` resolution logic, and the clipboard polling fallback path.

Single GPU tested. All measurements in this paper derive from a single NVIDIA RTX 3080 with 10 GB VRAM. Multi-GPU systems, NVIDIA RTX 40-series, AMD GPUs, and Apple Silicon have not been characterized. CTranslate2 supports multi-GPU via the `device_index` parameter; this is a configuration, not a tested deployment.

Translation quality not measured against human references.

The §5.5 beam-quality evaluation uses CTranslate2's model log-probability as a quality proxy — higher probability is a reasonable but imperfect signal for actual translation quality. We have not run a human evaluation or computed BLEU/chrF against a reference corpus. The §5.5 claim that beam=4 is "better" than beam=1 means "produces higher-probability outputs

under the model," not "produces outputs that human raters prefer." For a production deployment, a human evaluation on a representative gaming-chat corpus would be the appropriate follow-up.

The CUDA bug fix is reactive, not preventive. The fix in §4 ensures that the silent fallback is loud when it occurs and that the runtime recovers gracefully. It does not prevent the underlying DLL search path issue from arising in future CTranslate2 versions or in alternative Python CUDA libraries; the failure mode is a property of the Windows frozen-executable deployment model and is likely to recur.

Pre-release status. BabelGG v2 is currently in pre-release. The audit reports (`AUDIT_FIXING_PLAN.md` , `RELEASE_FIX_PLAN.md` , `PUBLISHING_READINESS_AUDIT_2026-04-13.md`) indicate ongoing engineering work; this paper describes the system as of the working tree at the time of writing, not as a shipping product.

Anti-virus and corporate-endpoint environments. The §4.10 broad-exception handling around `os.add_dll_directory` makes the fix robust against security software that blocks DLL loading, but also makes AV-related GPU initialization failures silent. On a corporate endpoint with aggressive DLL interception, the fix may fail to take effect without any user-visible symptom — a particularly insidious case.

NLLB linear translation weakness on long context. NLLB-200 produces sentence-level translations and does not maintain context across a multi-sentence input. A typical clipboard event may contain 2-5 sentences worth of game chat; the engine translates the concatenated text as a single sequence, which is acceptable for chat but degrades for narrative-style text.

7. Conclusion

Edge neural machine translation is viable today on consumer Windows hardware. The NLLB-200 distilled 600M model, wrapped in CTranslate2 with int8_float16 quantization on GPU, fits in ~1.2 GB VRAM on a consumer GPU class and translates in approximately 67 ms median in our benchmark corpus — fast enough for real-time gaming overlay use. The same model on the CPU fallback path achieves approximately 250 ms median latency, a 3.7× penalty that is meaningful but bounded.

The contribution of this paper is the diagnosis and fix of a previously undocumented failure mode in the deployment of CTranslate2 with CUDA support inside a frozen Windows PyInstaller executable: the engine silently falls back to CPU when the bundled CUDA runtime DLLs are not discoverable through the Windows DLL search path, producing correct translations at degraded latency with no diagnostic surface. The fix is a PyInstaller runtime hook plus an explicit `os.add_dll_directory` enumeration, totaling roughly 80 lines of Python. We document the failure mode, isolate the two interacting root causes, present the fix, and describe a two-stage verification system that catches the failure at both startup and runtime.

The finding matters for any developer shipping CTranslate2-based translation products through PyInstaller on Windows. The fix is small, well-localized, and does not require modifying CTranslate2 itself. We hope that by documenting the failure mode and the fix, future deployments of CTranslate2 on Windows will not silently fall back to CPU and present the resulting performance penalty as a hardware limitation rather than a packaging bug.

We have also surfaced, more prominently than we expected, a second engineering finding: the engine's source-language detector fails on a large fraction of European-language inputs because the detection pipeline relies on charset analysis (which cannot distinguish Latin-script languages from each other) and a small marker-word heuristic (which covers only the most common greeting and politeness words). When the detector fails, the engine returns `None` without ever invoking the translation model — the user sees a silent miss, not a poor translation. The fix is detection engineering: a 30-line `fasttext-langdetect` pre-detection step that raises detection accuracy from 51% to 86% and reduces the European → English failure rate from 50-85% to 0-20%.

The 600M parameter scale of NLLB is the practical ceiling for 10 GB consumer GPUs; users with higher quality requirements and better hardware should consider the 1.3B distilled variant on a 16 GB+ card, or the full 3.3B model on datacenter-class hardware. With the detection fix in place, the 600M variant is more capable on European-language inputs than the §5.2 results initially suggested.

BabelGG v2 is not a shipping product. It is a working engineering artifact demonstrating that the deployment is tractable, and exposing the failure modes that any developer attempting the same deployment is likely to encounter. We make the code, the benchmark harness, and the fix available in the accompanying repository.

Appendix A — Verification Notes

All code references in this paper were verified against the working tree at `D:\AbeSoft\BabelGG_v2` on 2026-08-05.

Hardware and runtime environment verified live:

- `nvidia-smi` — NVIDIA GeForce RTX 3080, 10 GB, driver 13.3
- `python -c "import ctranslate2; print(ctranslate2.__version__)"` via the project's `.venv` — 4.7.1;
`ctranslate2.get_cuda_device_count() == 1`
- CUDA toolkit 13.2 in `PATH`
(`/c/Program Files/NVIDIA GPU Computing Toolkit/CUDA/v13.2/bin`)
- `installer_build/app/_internal/nvidia/cublas/bin/cublas64_12.dll` exists, 102 MB, CUBLAS version 12.9
- `models/nllb-ct2-cpu/model.bin` — 622 MB
- `models/nllb-ct2-gpu/model.bin` — 619 MB
- `nvidia-smi` after model warmup — ~1,210 MiB used

Benchmark methodology: `bench.py` (in the repository) loads `FlashEngine` once per device, runs 10 language pairs × 20 sentences × 5 runs after one warmup throwaway call per sentence. Calls returning `None` are excluded from latency statistics but counted in the failure rate. Beam-size sweep uses a fresh engine per beam size with monkey-patched `_ct2_generate_text` to override the `beam_size` kwarg. Raw output in `bench_results.json` (CPU and GPU runs stored separately; rerun to refresh).

Failure rates in §5.2 are derived from the same benchmark and should be considered part of the deployment's measured characteristics rather than as test artifacts. The pattern — European → English rejected at 50-85%, English → CJK rejected at 0% — is consistent across runs and across CPU and GPU backends. The §5.8 detection fix changes this pattern materially: the same matrix run with fasttext-based detection drops the European → English rejection rate to 0-20%, confirming that the underlying NLLB model is capable on these inputs when given the correct source language tag.

Appendix B — fasttext-langdetect Integration

The full implementation of the detection fix described in §5.8. This wrapper sits ahead of the existing `detect_lang()` heuristic in `core/flash.py:731-784` and is invoked first; it falls back to

the original detector when fasttext is unavailable or returns low confidence.

```
# core/langid.py – BabelGG v2 detection fix
import ftlangdetect

_ISO_TO_FLASH = {
    'ja': 'jpn_Jpan', 'ko': 'kor_Hang', 'zh': 'zho_Hans', 'zh-cn': 'zho_Hans',
    'ar': 'arb_Arab', 'fr': 'fra_Latn', 'es': 'spa_Latn', 'de': 'deu_Latn',
    'pt': 'por_Latn', 'ru': 'rus_Cyrl', 'th': 'tha_Thai', 'vi': 'vie_Latn',
    'id': 'ind_Latn', 'tr': 'tur_Latn', 'it': 'ita_Latn', 'nl': 'nld_Latn',
    'pl': 'pol_Latn', 'sv': 'swe_Latn', 'hi': 'hin_Deva', 'en': 'eng_Latn',
}

_CONFIDENCE_THRESHOLD = 0.60

def detect_with_fasttext(text: str) -> tuple[str, float]:
    """Return (BCP-47 code, confidence). Falls back to (eng_Latn, 0.0)."""
    cleaned = (text or '').strip()
    if not cleaned:
        return 'eng_Latn', 0.0
    try:
        result = ftlangdetect.detect(text=cleaned, low_memory=True)
        iso = result.get('lang', 'unknown')
        score = float(result.get('score', 0.0))
        flash = _ISO_TO_FLASH.get(iso, 'eng_Latn')
        if score >= _CONFIDENCE_THRESHOLD:
            return flash, score
    except Exception:
        pass
    return 'eng_Latn', 0.0
```

The wrapper is then prepended to the existing

FlashEngine.detect_lang method:

```
def detect_lang(self, text: str) -> str:
    # Pre-detection via fasttext (returns early if high-confidence)
    ft_code, ft_score = detect_with_fasttext(text)
    if ft_score >= _CONFIDENCE_THRESHOLD and (
        ft_code in self._DETECT or ft_code in (
            'fra_Latn', 'spa_Latn', 'deu_Latn', 'por_Latn', 'ita_Latn',
            'nld_Latn', 'pol_Latn', 'swe_Latn', 'rus_Cyrl', 'hin_Deva',
            'tur_Latn', 'vie_Latn', 'ind_Latn',
        )
    ):
        self._last_detect_reason = f'ft:{ft_code}({ft_score:.2f})'
        return ft_code
    # Fall through to existing heuristic-based detection
    return self._legacy_detect_lang(text)
```

The full integration is approximately 30 lines of Python. The detection call adds approximately 1-2 ms per call on this hardware, well within the soft-budget constant of 600 ms (core/flash.py:153), and is invoked once per translation request.

To enable the fix in the deployment:

1. Add `fasttext-langdetect` to `requirements.txt` (the package includes the pre-trained fastText language identification model).
2. Add `core/langid.py` with the wrapper shown above.
3. Replace `core/flash.py: detect_lang` with the prepended version.
4. No re-quantization of the NLLB model is required; the fix is entirely in the detection preprocessing step.